Diffusion Acceleration

Deep Dive into SGLang — Chapter 32

Yin Yang

Foundation Books Project

Where this chapter sits

Chapter 31 turned a diffusion request into a request-owned continuation record. This chapter asks: how do we make that record *cheaper* to serve without changing what it represents?

- Every shortcut — offload a VAE, reuse a residual, swap an attention kernel, quantize the denoiser — can make the same request fit or run faster.
- Every shortcut also creates a new way to run the *wrong* component, reuse a *stale* cache, or compare a warm path against a cold baseline.
- One trace carries the chapter: the Qwen-Image request from Chapter 31 — "a red mug on a wooden desk," seed 1337, 1024×1024, 30 denoising steps.

Acceleration is fixed-request accounting: the accelerated path is valid only when its records keep the comparison honest.

Roadmap

The fixed-request problem

Local records

Timeline and shortcuts

Invariant and evidence

The fixed-request problem

Same trajectory, less cost

Diffusion acceleration reduces end-to-end media cost while producing the *same* requested denoising trajectory. The baseline loop is fixed:

$$\text{encode prompt} \;\rightarrow\; \text{for } t = 30, \ldots, 1: \; z_{t-1} = \Phi_\theta(z_t, c, t) \;\rightarrow\; \text{VAE decode once.}$$

- Keeping every component resident is easy — but it can make larger models impossible to serve.
- Offload, reuse, faster kernels, or quantization can make the same request fit; the price is extra runtime state that must stay *aligned* with the latent trajectory.
- Fixed: prompt, seed, latent shape, timestep schedule, output size. Free to change: component residency, overlap timing, kernel, precision.

Acceleration changes placement and reuse *around* the loop — not which request-owned trajectory it advances.

Local records

Four records make acceleration auditable

Record	What it pins down
Acceleration policy π_{acc}	Which rule changed: cache, residency, offload, precision, backend, compile, benchmark mode.
Component residency $\rho_t(\gamma)$	Placement, availability, next use, and the policy action attached to that next use.
Semantic cache-reuse key K_{sem}	The metadata proof that a cached result belongs to the current step.
Residency/offload/cache invariant	The gate: ready, fresh, applicable before a step consumes optimized state.

The records identify the execution path; the invariant decides whether that path is legal for this request, this step.

The cost account: where does time go?

A residency policy adds movement terms around the model work:

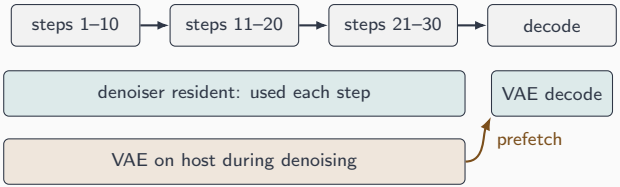
$$T_{\text{image}} = \sum_{t \in \mathcal{T}} T_t + T_{\text{prefetch}} + T_{\text{wait}} + T_{\text{release}} + T_{\text{decode}}.$$

- $\sum_t T_t$ is logical model work; prefetch, wait, and release are implementation work the policy introduced.
- The goal is *not* to shrink every term — a policy may add prefetch to reduce peak memory enough to run a larger model.
- A benchmark that reports only final image latency misses which term moved.

Comparing two runs is meaningful only when every term names the same work under a fixed protocol.

Timeline and shortcuts

Residency is a timeline, not a CPU/GPU toggle



- The manager collects the *ordered* component uses, prepares each before its stage, and prefetches the next memory-intensive use.
- Layerwise offload applies the same contract *inside* the DiT: a layer's weights must be resident before its forward hook.

Prefetch the next component only when it is the next semantic use.

Reuse, attention, and precision run inside the call

- **Reuse.** TeaCache asks whether adjacent denoising states are similar enough to reuse a residual; Cache-DiT asks which blocks and steps may reuse work. A hit is valid only when keyed by component, step, latent, conditioning, precision, and policy state.
- **Attention.** A backend may change *how* attention is computed — FlashAttention, Sage, sparse-video, sequence/ring-parallel — but not *which* tokens, frames, or masks are attended to.
- **Precision.** Nunchaku SVDQuant stores the transformer in a low-rank-plus-quantized form; that precision policy becomes part of component identity for every later step.

Each is legal only when it preserves the request's latent transition.

Invariant and evidence

The gate on optimized execution

Invariant (Residency/offload/cache)
Before component χ runs for request r, the optimized record must match the current request state:

$$\text{consistent}\big((v_\chi, d_\chi, k_\chi, p_\chi), (z_{r,t}, c_r, \tau_t)\big).$$

- **Ready**: the module/layer is callable on the device that runs it.
- **Fresh**: weights and buffers match the selected component version.
- **Applicable**: any cached state was produced under compatible timestep, latent, conditioning, and policy metadata.

A failure in any part is silent — shape stays correct while the trajectory diverges.

A claim is comparable only if it changes one surface

Claim	Changed policy	Held fixed / reported
VAE delayed decode	Decoder residency, transfer timing	Trajectory, latent shape, VAE options, output size
Layerwise DiT offload	Per-layer prefetch & release	DiT config, scheduler, backend, precision, hardware
Cache reuse	TeaCache / Cache-DiT rule	Conditioning, trajectory, region, branch, family
Attention / quant	Kernel or representation	Tokens, mask, tolerance, hardware eligibility

If a variant also changes the scheduler, cache, or quantization, the row is no longer evidence for the one surface it claims.

What to carry forward

1. Acceleration is **fixed-request accounting**: same trajectory, cheaper path.
2. Four records — **policy**, **residency**, **cache key**, **invariant** — make a shortcut auditable.
3. Residency is a **timeline**; offload and layerwise offload obey one prepare-before-use contract.
4. Reuse, attention, and precision are valid only when they preserve the request's latent transition.
5. A speedup is often **cost moved**, not work erased — so compare only under a fixed protocol.

Next: benchmarking methodology — fixing the workload and protocol so an acceleration claim can actually be measured.